

Security Audit

Stoneii 2nd (DeFi)

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	6
Security Rating	7
Intended Smart Contract Functions	8
Code Quality	11
Audit Resources	11
Dependencies	11
Severity Definitions	12
Status Definitions	13
Audit Findings	14
Centralisation	36
Conclusion	37
Our Methodology	38
Disclaimers	40
About Hashlock	41

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The Stoneii team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

Stoneii Smart is a smart-contract suite designed to support compliant on-chain fundraising and private-equity style token offerings on EVM networks. The system includes infrastructure for token presales, private equity issuance, fee management, and admin recovery, alongside a KYC Soulbound token to enforce identity and eligibility requirements. By combining routers, factories, and compliance modules, the project aims to provide a structured, regulation-aware framework for launching and managing tokenized investment products in a secure and transparent way.

Project Name: Stoneii

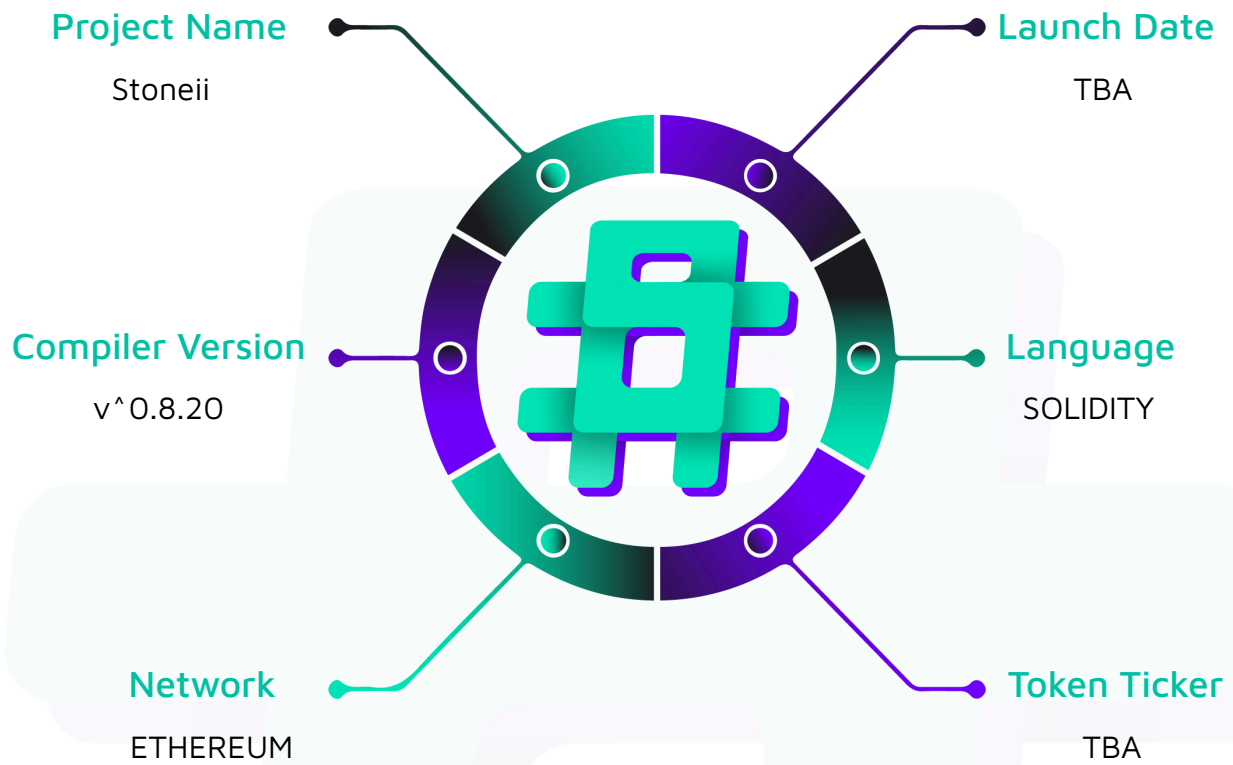
Project Type: Defi

Compiler Version: ^0.8.20

Website: <https://www.stoneii.com/>

Logo:



Visualised Context:

Audit Scope

We at Hashlock audited the solidity code within the Stoneii project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	Stoneii Smart Contracts
Network	Ethereum
Language	Solidity
Audit Date	March, 2026
Contract 1	KYCSoulBound.sol
Contract 2	PresaleRouterContract.sol
Contract 3	PresaleFactory.sol
Contract 4	AdminRescue.sol
Contract 5	Compliance.sol
Contract 6	FeeManager.sol
Contract 7	PrivateEquityRouter.sol
Contract 8	PrivateEquityToken.sol
Audited GitHub Commit Hash	ad255bb3b4c1bdca7f8a2dc45e54c23503eb546d
Fix Review GitHub Commit Hash	e112566ab1997b0d8eedc3cbdd5c13c7a5d2d490

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.



The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

5 High severity vulnerabilities

2 Medium severity vulnerabilities

4 Low severity vulnerabilities

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
KYCSoulBound.sol - Allows users to: - Mint a KYC SBT for a user - Check SBT ownership status - Allows admins to: - Revoke a user's SBT - Add or remove admin roles	Contract achieves this functionality.
PresaleRouterContract.sol - Allows users to: - Buy presale tokens with USDC - Claim tokens after success - Claim refunds after failure - Allows admins to: - Finalize presale success or failure - Create the real token via router	Contract achieves this functionality.
PresaleFactory.sol - Allows users to: - View deployed presale contracts - Allows admins to: - Deploy new presale contracts - Set the router address - Configure admin fee recipient	Contract achieves this functionality.
AdminRescue.sol - Allows users to: - None	Contract achieves this functionality.

- Allows admins to: - Rescue ETH from the contract - Rescue ERC20 tokens from the contract	
Compliance.sol - Allows users to: - Check SBT ownership status - Query transfer/mint/burn allowance - Allows admins to: - Update router address - Set whitelist entries - Set SBT contract address	Contract achieves this functionality.
FeeManager.sol - Allows users to: - None - Allows admins to: - None	Contract achieves this functionality.
PrivateEquityRouter.sol - Allows users to: - Buy tokens with USDC - Redeem tokens for USDC - Trigger annual fee payments - Allows admins to: - Create and configure new tokens - Update price, cap, minting, redeem status - Manage whitelist and SBT address - Fund redemption liquidity	Contract achieves this functionality.
PrivateEquityToken.sol - Allows users to: - Transfer tokens (with router checks) - TransferFrom tokens (with router checks)	Contract achieves this functionality.

- Allows admins to: - Mint tokens - Burn tokens - Update token cap	
---	--



Code Quality

This audit scope involves the smart contracts of the Stoneii project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring were recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Stoneii project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-severity issues should be resolved before deployment. While they do not typically lead to a complete loss of funds, they may result in partial loss of funds or unintended behavior under certain conditions.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] Compliance#isTransferAllowed - Burn revert cascades across protocol

Description

The `Compliance.isTransferAllowed()` function had its explicit burn handler (`to == address(0)`) commented out when subscription logic was removed. The SBT gating check that follows now unconditionally calls `sbt.balanceOf(address(0))`, which reverts in OpenZeppelin v5 ERC721 with `ERC721InvalidOwner`. This breaks every burn-path operation in the protocol: `buyTokens` with fees, `redeem`, and annual fee collection all become permanently non-functional.

Vulnerability Details

When subscription logic was removed, lines 76–82 in `Compliance.sol` were commented out. These lines contained the early return for burn operations (`to == address(0) → return true`). Without this handler, burns fall through to the SBT gating check at lines 97–98:

```
// Compliance.sol - SBT gating (lines 97-101)
if (address(sbt) != address(0)) {
    if (sbt.balanceOf(from) == 0 || sbt.balanceOf(to) == 0) {
        revert SBTRequired(from, to);
    }
}
```

When `to == address(0)` (burn), `sbt.balanceOf(address(0))` is called. `KYCSoulBound` inherits OZ v5 `ERC721`, whose `balanceOf(address(0))` reverts with `ERC721InvalidOwner(address(0))`.

The revert cascades through every burn call chain:

- `buyTokens()` → `token.burn()` → `_update(from, address(0))` → `beforeTokenTransfer` → `compliance.isTransferAllowed(id, from, address(0))` → REVERT
- `redeem()` → explicit compliance check with `to=address(0)` → REVERT
- `_collectAnnualFee()` → `token.burn()` → same chain → REVERT

Proof of Concept

```
function test_buyTokens_burnReverts() public {
    vm.startPrank(user1);
    usdc.approve(address(router), 1_000e6);
    vm.expectRevert();
    router.buyTokens(tokenId, 10e18, 20e6, block.timestamp + 1000, false);
    vm.stopPrank();
}

function test_redeem_burnReverts() public {
    router.mintToWallet(tokenId, user1, 10e18);
    vm.startPrank(user1);
    vm.expectRevert();
    router.redeem(tokenId, 5e18, 0, block.timestamp + 1000, false);
    vm.stopPrank();
}

function test_sbt_balanceOfZero_reverts() public {
    vm.expectRevert();
    sbt.balanceOf(address(0));
}
```

Impact

All token burn operations are permanently blocked, rendering the core protocol non-functional. Users cannot purchase tokens with transaction fees enabled because the fee burn reverts. Users cannot redeem tokens for stablecoins because the burn and compliance check both revert. Annual fee collection is broken because it requires burning fee tokens. The protocol is effectively frozen with no admin workaround — the



commented-out burn handler must be restored for any burn-dependent functionality to operate.

Recommendation

Restore the burn handler in `Compliance.isTransferAllowed()` before the SBT gating check:

```
// Compliance.sol - isTransferAllowed()
+ // BURN - skip SBT check for burns
+ if (to == address(0)) {
+     return true;
+ }
+
// SBT gating
if (address(sbt) != address(0)) {
    if (sbt.balanceOf(from) == 0 || sbt.balanceOf(to) == 0) {
```

Status

Resolved

[H-02] PrivateEquityRouter#constructor - USDT self-assignment breaks all USDT operations

Description

In the `PrivateEquityRouter` constructor, the USDT state variable is assigned to itself instead of the constructor parameter. Line 141 reads `usdtToken = IERC20(usdtToken)` (referencing the uninitialized state variable) instead of `usdtToken = IERC20(_usdtToken)` (the constructor parameter). This leaves `usdtToken` permanently set to `address(0)`, breaking every USDT-denominated operation in the protocol.

Vulnerability Details

The constructor correctly assigns USDC on line 140 but contains a copy-paste typo on line 141:

```
// PrivateEquityRouter.sol - constructor
usdcToken = IERC20(_usdcToken); // line 140 - CORRECT: uses parameter _usdcToken
usdtToken = IERC20(usdtToken);   // line 141 - BUG: uses state var usdtToken
                                   (address(0))
```

The constructor parameter is `_usdtToken` (with underscore), but the assignment references the state variable `usdtToken` (without underscore). Since state variables default to `address(0)` before initialization, `usdtToken` is permanently set to `IERC20(address(0))`. No setter function exists to correct this post-deployment.

Every function that branches on `useUSDT == true` resolves `usdtToken` to `address(0)`:

- `buyTokens()` → `safeTransferFrom(address(0), ...)` → revert
- `redeem()` → `balanceOf(address(0))` / `safeTransfer(address(0), ...)` → revert
- `adminFundStable()` → `safeTransferFrom(address(0), ...)` → revert
- `depositDividends()` → `safeTransferFrom(address(0), ...)` → revert
- `createToken()` propagates `address(0)` to `PrivateEquityToken` → USDT dividends broken

Impact

The entire USDT integration added in this update is non-functional. Users cannot buy tokens with USDT, cannot redeem tokens for USDT, and administrators cannot fund the router with USDT or deposit USDT dividends. Since the protocol was explicitly updated to support dual stablecoin (USDC + USDT) operations, half of the payment infrastructure is broken. There is no on-chain workaround — redeployment is required.

Recommendation

Fix the typo on line 141 of the constructor:

```
// PrivateEquityRouter.sol - constructor
usdcToken = IERC20(_usdcToken);
- usdtToken = IERC20(usdtToken);
+ usdtToken = IERC20(_usdtToken);
```

Status

Resolved

[H-03] PrivateEquityToken#_update - Dividend debt not reset on transfer enables theft and insolvency

Description

The `PrivateEquityToken._update()` override calls `_updateDividends()` for both sender and receiver before `super._update()` executes the balance change, but only re-initializes debt for mints (`from == address(0)`). On regular transfers, the receiver's debt is set based on their pre-transfer balance, leaving a gap equal to the transferred amount's accumulated dividends. This allows the receiver to claim dividends they never earned, leading to protocol insolvency.

Vulnerability Details

In `_update()` (lines 120–142), the call ordering creates the vulnerability:

```
// PrivateEquityToken.sol - _update()
if (from != address(0)) _updateDividends(from);
if (to != address(0)) _updateDividends(to);    // reads balanceOf(to) = OLD balance

super._update(from, to, amount);              // balance changes HERE

// Debt only re-initialized for MINTS
if (from == address(0)) {
    usdcDebt[to] = (balanceOf(to) * accUsdcPerToken) / ACC_PRECISION;
    usdtDebt[to] = (balanceOf(to) * accUsdtPerToken) / ACC_PRECISION;
}
```

`_updateDividends(to)` computes debt using `balanceOf(to)` which is the pre-transfer balance. After `super._update()`, the receiver holds `oldBalance + amount` tokens, but their debt only accounts for `oldBalance`. The difference `(amount * accPerToken) / PRECISION` becomes phantom claimable dividends. There is no post-transfer debt correction for regular transfers — only mints get the correction at lines 138–141.

Proof of Concept

```
function test_dividend_theft_via_transfer() public {
    // Alice holds 1000 tokens, 100 USDC dividends deposited
    vm.prank(alice);
```



```

    peToken.transfer(bob, 500e18);

    // Bob's debt is 0 - set when he had 0 balance (pre-transfer)
    assertEq(peToken.usdcDebt(bob), 0);

    uint256 alicePending = peToken.pendingUsdc(alice);

    // Bob claims 50 USDC he never earned
    vm.prank(bob);
    peToken.claimDividends();
    assertEq(usdc.balanceOf(bob), 50e6);

    // Contract is now insolvent - cannot pay Alice's 100 USDC
    assertLt(usdc.balanceOf(address(peToken)), alicePending);

    // Alice's claim reverts
    vm.prank(alice);
    vm.expectRevert();
    peToken.claimDividends();
}

```

Impact

Any token transfer enables the receiver to claim dividends for all prior distribution rounds on the received tokens, even though those dividends belong to the sender. The total claimable dividends across all holders exceeds the actual deposited amount, creating protocol insolvency. The last users to claim will have their transactions revert due to insufficient contract balance. With a single transfer of 50% of supply, an attacker can steal 50% of all deposited dividends and permanently lock the remaining holders out of their rightful claims.

Recommendation

Re-initialize the receiver's debt after `super.\_update()` for all transfer types, not just mints:

```

super._update(from, to, amount);

- if (from == address(0)) {

```

```
+ if (to != address(0)) {  
    usdcDebt[to] = (balanceOf(to) * accUsdcPerToken) / ACC_PRECISION;  
    usdtDebt[to] = (balanceOf(to) * accUsdtPerToken) / ACC_PRECISION;  
}
```

Status

Resolved



[H-04] Compliance#isTransferAllowed - walletBlacklist mapping written but never enforced

Description

The `walletBlacklist` mapping is written by `setwalletBlacklist()` and exposed via Router's `blacklistWallet()` / `unblacklistWallet()` admin functions, but `isTransferAllowed()` never reads it. Blacklisted addresses retain full protocol access — they can buy, sell, transfer, and receive tokens without restriction. The regulatory compliance feature is completely non-functional.

Vulnerability Details

The `walletBlacklist` mapping was added at `Compliance.sol:23` with a setter at lines 46–48. The Router exposes admin functions `blacklistWallet()` (line 279) and `unblacklistWallet()` (line 283) confirming this is an intended security feature. However, `isTransferAllowed()` (lines 59–105) never references the mapping:

```
// Compliance.sol
mapping(address => bool) public walletBlacklist; // line 23 - declared

function setwalletBlacklist(address who, bool status) external onlyRouter {
    walletBlacklist[who] = status; // line 47 - written
}

function isTransferAllowed(uint256 id, address from, address to)
    external view returns (bool)
{
    if (dexWhitelist[from] || dexWhitelist[to]) return true; // line 65
    // ... SBT checks ...
    // walletBlacklist is NEVER checked anywhere in this function
    return true;
}
```

A codebase-wide search confirms zero reads of `walletBlacklist` outside of the setter and the public getter. The mapping is dead state.

Proof of Concept

```
function test_blacklisted_canBuy() public {
    router.blacklistWallet(badActor);
    assertTrue(compliance.walletBlacklist(badActor));

    vm.startPrank(badActor);
    usdc.approve(address(router), 100e6);
    router.buyTokens(0, 10e18, 100e6, block.timestamp + 1 hours, false);
    vm.stopPrank();

    (address tokenAddr,,,,,,,,) = router.tokenConfigs(0);
    assertGt(IERC20(tokenAddr).balanceOf(badActor), 0);
}

function test_blacklisted_canTransfer() public {
    vm.startPrank(badActor);
    usdc.approve(address(router), 100e6);
    router.buyTokens(0, 10e18, 100e6, block.timestamp + 1 hours, false);
    vm.stopPrank();

    router.blacklistWallet(badActor);

    (address tokenAddr,,,,,,,,) = router.tokenConfigs(0);
    vm.prank(badActor);
    IERC20(tokenAddr).transfer(recipient, 5e18);
    assertEq(IERC20(tokenAddr).balanceOf(recipient), 5e18);
}
```

Impact

The blacklist feature advertised by the protocol's admin interface is entirely non-functional. Sanctioned, malicious, or compromised addresses flagged by the administrator retain full, unrestricted access to all token operations. For a regulated private equity token platform, this represents a complete failure of the compliance enforcement layer. Administrators operating under the belief that blacklisting is active face regulatory liability, as flagged addresses continue transacting without any on-chain restriction.

Recommendation

Add a blacklist check at the beginning of `isTransferAllowed()`:

```
function isTransferAllowed(uint256 id, address from, address to)
    external view returns (bool)
{
+   if (walletBlacklist[from] || walletBlacklist[to]) {
+       revert Blacklisted(from, to);
+   }
+
    if (dexWhitelist[from] || dexWhitelist[to]) return true;
```

Status

Resolved

[H-05] PresaleRouterContract#settleSuccess -Interface parameter mismatch breaks presale settlement

Description

The Presale contract's `IPrivateEquityRouterView` declares `createToken` with 10 parameters, but the actual `PrivateEquityRouter.createToken` requires 11 parameters after the `usdfreeReceipient` field was added. Different parameter counts produce different function selectors, so `settleSuccess()` always reverts. No presale can ever be finalized as successful.

Vulnerability Details

The interface mismatch is between two copies of `IPrivateEquityRouterView` in separate directories:

```
// Presale interface - 10 params (PresaleToken/IPrivateEquityRouterView.sol:39-50)
function createToken(
    string memory, string memory, uint256, uint256,
    bool, uint256, uint256, uint256, bool,
    address          // single feeRecipient
) external returns (uint256);
// Selector: 0xf77aab28

// Router actual - 11 params (PrivateEquity/PrivateEquityRouter.sol:158-170)
function createToken(
    string memory, string memory, uint256, uint256,
    bool, uint256, uint256, uint256, bool,
    address,          // tokenfeeRecipient
    address          // usdfreeReceipient ← ADDED
) external returns (uint256);
// Selector: 0xdb2001ce
```

When `settleSuccess()` calls `router.createToken(...)` with 10 arguments, the EVM looks up selector `0xf77aab28` in the Router. The Router only has selector `0xdb2001ce` (11 params) and no fallback function, so the call reverts.

Proof of Concept

```
function test_settleSuccess_selectorMismatch() public {
    usdc.mint(buyer, 100e6);
    vm.startPrank(buyer);
    usdc.approve(address(presale), 100e6);
    presale.buy(100e6, false);
    vm.stopPrank();

    assertEq(presale.totalRaised(), 100e6);

    // Reverts: Presale encodes 10-param selector, Router expects 11-param
    vm.expectRevert();
    presale.settleSuccess(
        keccak256(abi.encode("dummy")),
        "TestToken", "TT", 1000e18, 1e6,
        true, 100, 0, 0, false, feeRecipient
    );

    // Presale stuck in Active - cannot finalize
    assertEq(uint256(presale.status()), uint256(Presale.PresaleStatus.Active));
}

function test_createToken_selectorDiffers() public pure {
    bytes4 presaleSelector = bytes4(keccak256(
        "createToken(string,string,uint256,uint256,bool,uint256,uint256,uint256,bool,address)"
    ));
    bytes4 routerSelector = bytes4(keccak256(
        "createToken(string,string,uint256,uint256,bool,uint256,uint256,uint256,bool,address,address)"
    ));
    assertTrue(presaleSelector != routerSelector);
}
```

Impact

The entire presale-to-token lifecycle is broken. No presale can ever be settled as successful, meaning token creation through the presale flow is impossible. Users who



contributed USDC/USDT during presales cannot claim their tokens because the presale never reaches 'Successful' status. The 'settleFail()' function still works as a recovery path for refunds, so funds are not permanently locked, but the presale feature is completely non-functional.

Recommendation

Update the Presale's 'IPrivateEquityRouterView' to match the Router's 11-parameter signature:

```
// PresaleToken/IPrivateEquityRouterView.sol
function createToken(
    string memory, string memory, uint256, uint256,
    bool, uint256, uint256, uint256, bool,
-   address
+   address,
+   address
) external returns (uint256);
```

Also update 'settleSuccess()' to pass the additional 'usdfeeReceipient' parameter.

Status

Resolved

Medium

[M-01] FeeManager#_accruedAnnualFee - Fee evasion for presale and admin-minted tokens

Description

`lastFeeTimestamp` is only initialized in `buyTokens()`. Users who receive tokens via `mintToWallet()`, `mintFromPresale()`, or ERC20 transfer have `lastFeeTimestamp == 0`, causing `\_accruedAnnualFee()` to permanently return zero. This creates a permanent annual fee exemption for all non-`buyTokens` minting paths with no admin workaround.

Vulnerability Details

The fee accrual function contains an early return when the timestamp is uninitialized:

```
// FeeManager.sol - _accruedAnnualFee() (line 36)
if (lastTimestamp == 0 || annualFeeBps == 0) return 0;
```

Only `buyTokens()` sets this timestamp (lines 397–401). The three other token acquisition paths never initialize it:

- `mintToWallet()` (Router:241–251) — admin mint, no timestamp logic
- `mintFromPresale()` (Router:56–68) — presale claim, no timestamp logic
- ERC20 `transfer()` — `beforeTokenTransfer` is `view`, cannot mutate state

A dead-code function `\_initAnnualFeeIfNeeded()` (FeeManager:25–29) exists but is never called, confirming the omission was unintentional. The fee deadlock is permanent: `\_collectAnnualFee` returns early when fee is 0, so the timestamp update at line 67 is never reached.

Impact

Token holders who acquire tokens through presale claims, admin mints, or secondary transfers are permanently exempt from annual fees. This creates an uneven fee burden where only direct buyers pay annual fees, while all other holders evade them indefinitely. Over time, this results in significant protocol revenue loss and an unfair

advantage for non-buyer token holders, undermining the economic model of the private equity token system.

Recommendation

Call `_initAnnualFeeIfNeeded` in all minting paths, or initialize the timestamp in `_update()`:

```
// PrivateEquityToken.sol - _update()
super._update(from, to, amount);

if (from == address(0)) {
    usdcDebt[to] = (balanceOf(to) * accUsdcPerToken) / ACC_PRECISION;
    usdtDebt[to] = (balanceOf(to) * accUsdtPerToken) / ACC_PRECISION;
+   IPrivateEquityRouterView(router).initAnnualFeeIfNeeded(id, to);
}
```

Status

Resolved

[M-02] Contract - Presale SBT desynchronization traps user funds after migration

Description

`Router.setSBTContract()` updates the SBT reference in Router and Compliance but not in existing Presale contracts or PresaleFactory. If the SBT contract is migrated between a user's `buy()` and `claimTokens()` calls, the user passes `buy()` with the old SBT but fails `claimTokens()` → `mintFromPresale()` → `compliance.doesHaveSbt()` with the new SBT. Funds are trapped with no on-chain recovery.

Vulnerability Details

The SBT reference is stored independently in three locations with no synchronization:

```
// Router.setSBTContract() - only updates 2 of 3 references
function setSBTContract(address s) external onlyOwner {
    sbtContract = ISBT(s);           // Router - updated
    compliance.setSBT(s);           // Compliance - updated
    // Presale.sbt - NOT updated (no setter exists)
    // PresaleFactory.sbt - NOT updated (no setter exists)
}
```

The claim path crosses the desync boundary:

1. `Presale.buy()` (line 75): checks `IERC20(sbt).balanceOf(msg.sender)` — old SBT
2. `Presale.claimTokens()` (line 159): calls `router.mintFromPresale()`
3. `Router.mintFromPresale()` (line 61): checks `compliance.doesHaveSbt(to)` — new SBT
4. User fails the new SBT check → revert. Cannot claim tokens, cannot refund (status is `Successful`).

Impact

Users who participated in a presale before an SBT migration are permanently locked out of their token claims. They cannot claim tokens because they fail the new SBT check, and they cannot claim refunds because the presale status is `Successful`. The admin workaround of calling `withdrawUSD()` and manually distributing tokens is off-chain and

provides no on-chain guarantee. Additionally, `PresaleFactory.sbt` also lacks a setter, so all future presales deployed after migration will also use the stale SBT.

Recommendation

Add SBT setters to both Presale and PresaleFactory, and update them in `setSBTContract()`:

```
function setSBTContract(address s) external onlyOwner {  
    sbtContract = ISBT(s);  
    compliance.setSBT(s);  
+   presaleFactory.setSBT(s);  
}
```

Status

Resolved - `PresaleFactory.setSbt()` added. Remaining Presale-level desync is a non-issue as SBT contract is immutable post-deployment by design.

Low

[L-01] PrivateEquityRouter#buyTokens - Cost truncates to zero for dust amounts

Description

The cost calculation `costStable = (cfg.adminPriceUSDC * amount) / 1e18` truncates to zero when `adminPriceUSDC * amount < 1e18`. For a price of 1 USDC (`1e6`), any `amount < 1e12` yields zero cost, allowing free token minting. The economic impact is negligible — accumulating 1 full token requires ~1M transactions, with gas costs exceeding the token value by 3–6 orders of magnitude on any chain.

Recommendation

Add a minimum cost check after the calculation:

```
uint256 costStable = (cfg.adminPriceUSDC * amount) / 1e18;  
+ require(costStable > 0, "Amount too small");  
require(costStable <= maxCostStable, "Slippage exceeded");
```

Status

Resolved

[L-02] Contract - DEX whitelist bypass enables KYC circumvention via fee recipient

Description

`createToken()` auto-whitelists the `tokenfeeRecipient` via `compliance.setWhitelist()`. The DEX whitelist bypass at `Compliance.isTransferAllowed()` line 65 (`dexWhitelist[from] || dexWhitelist[to]`) skips ALL compliance checks when either party is whitelisted, including SBT/KYC gating. A whitelisted fee recipient can therefore transfer tokens to non-KYC addresses without restriction. Impact is limited because the fee recipient is a privileged address set by the owner, and only accumulated fee tokens are at risk.

Recommendation

Restrict the DEX whitelist bypass to only skip checks for the whitelisted party, not the counterparty:

```
- if (dexWhitelist[from] || dexWhitelist[to]) return true;
+ if (dexWhitelist[from] && dexWhitelist[to]) return true;
+ if (dexWhitelist[from]) {
+     if (address(sbt) != address(0) && sbt.balanceOf(to) == 0) revert SBTRequired(from, to);
+     return true;
+ }
```

Status

Acknowledged — intended behavior per team

[L-03] PrivateEquityRouter#claimDividends - Dividend claim uses Router as msg.sender instead of user

Description

`Router.claimDividends(tokenId)` calls `PrivateEquityToken.claimDividends()` without forwarding the user's address. Inside the token contract, `msg.sender` resolves to the Router address, which holds zero tokens and zero pending dividends. The function executes a no-op. Users can work around this by calling `PrivateEquityToken.claimDividends()` directly, since it has no access restriction. Dividends are not locked — the Router convenience function is simply broken.

Recommendation

Pass the caller's address to the token contract:

```
- function claimDividends(uint256 tokenId) external tokenExists(tokenId) {  
-     PrivateEquityToken(tokenConfigs[tokenId].tokenAddress).claimDividends();  
+ function claimDividends(uint256 tokenId) external tokenExists(tokenId) {  
+  
+     PrivateEquityToken(tokenConfigs[tokenId].tokenAddress).claimDividendsFor(msg.sender);  
+ }
```

Add a `claimDividendsFor(address user)` function to `PrivateEquityToken` with `onlyOwner` (Router) access control.

Status

Resolved

[L-04] PresaleRouterContract#withdrawUSD - No status check allows owner to drain during active presale

Description

`withdrawUSD()` has only an `onlyOwner` modifier with no `PresaleStatus` check. The owner can drain all USDC and USDT from the presale contract at any time — including during an active presale with user contributions. If the owner later calls `settleFail()`, users' `claimRefund()` calls revert due to zero contract balance, permanently locking their funds.

Recommendation

Add a status guard that restricts withdrawals to successful presales only:

```
function withdrawUSD(address to) external onlyOwner {  
+   require(status == PresaleStatus.Successful, "Presale not settled");  
    uint256 bal = usdc.balanceOf(address(this));  
    usdc.safeTransfer(to, bal);  
}
```

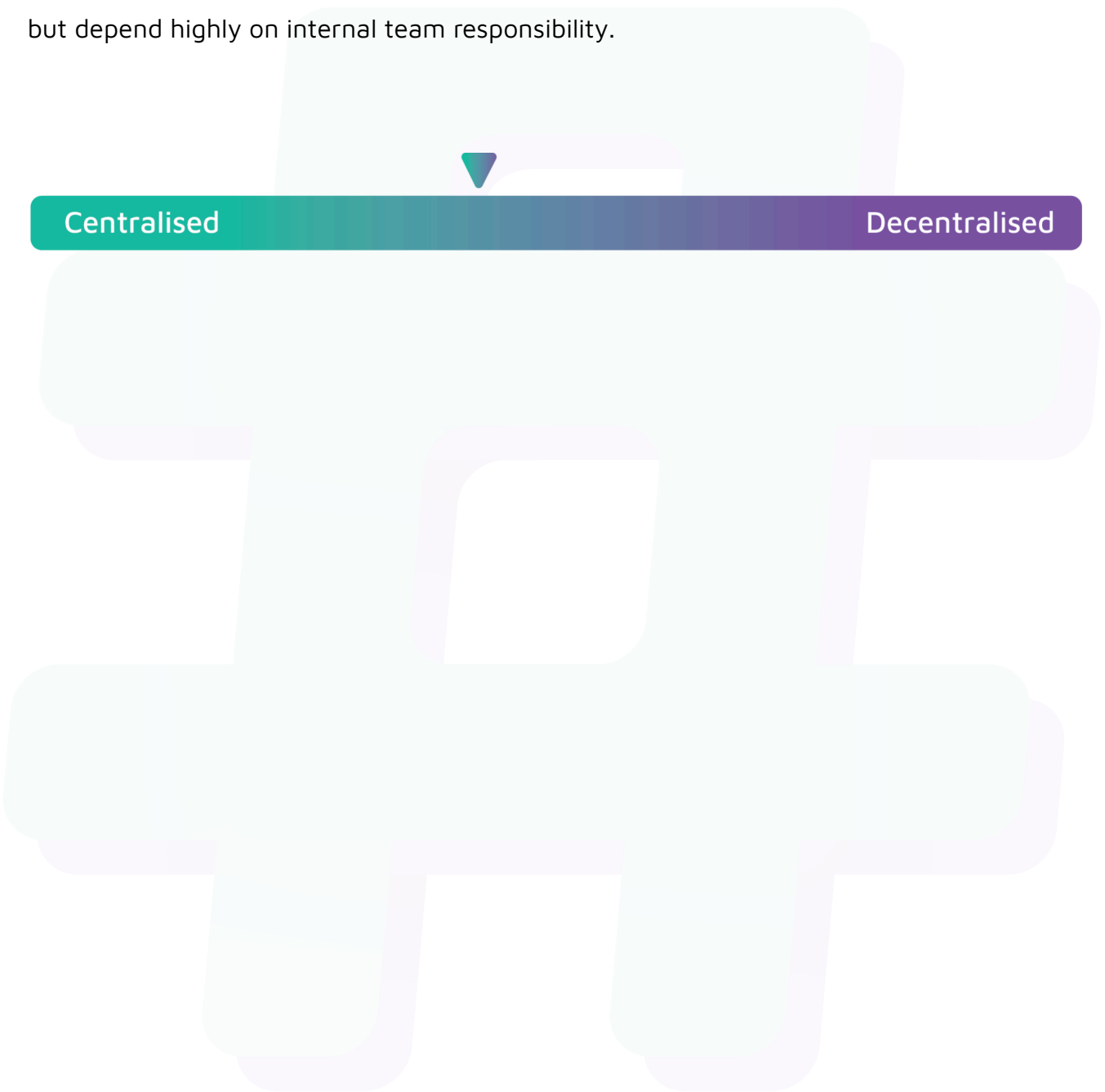
Status

Acknowledged — intended behavior per team

Centralisation

The Stoneii project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the Stoneii project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.